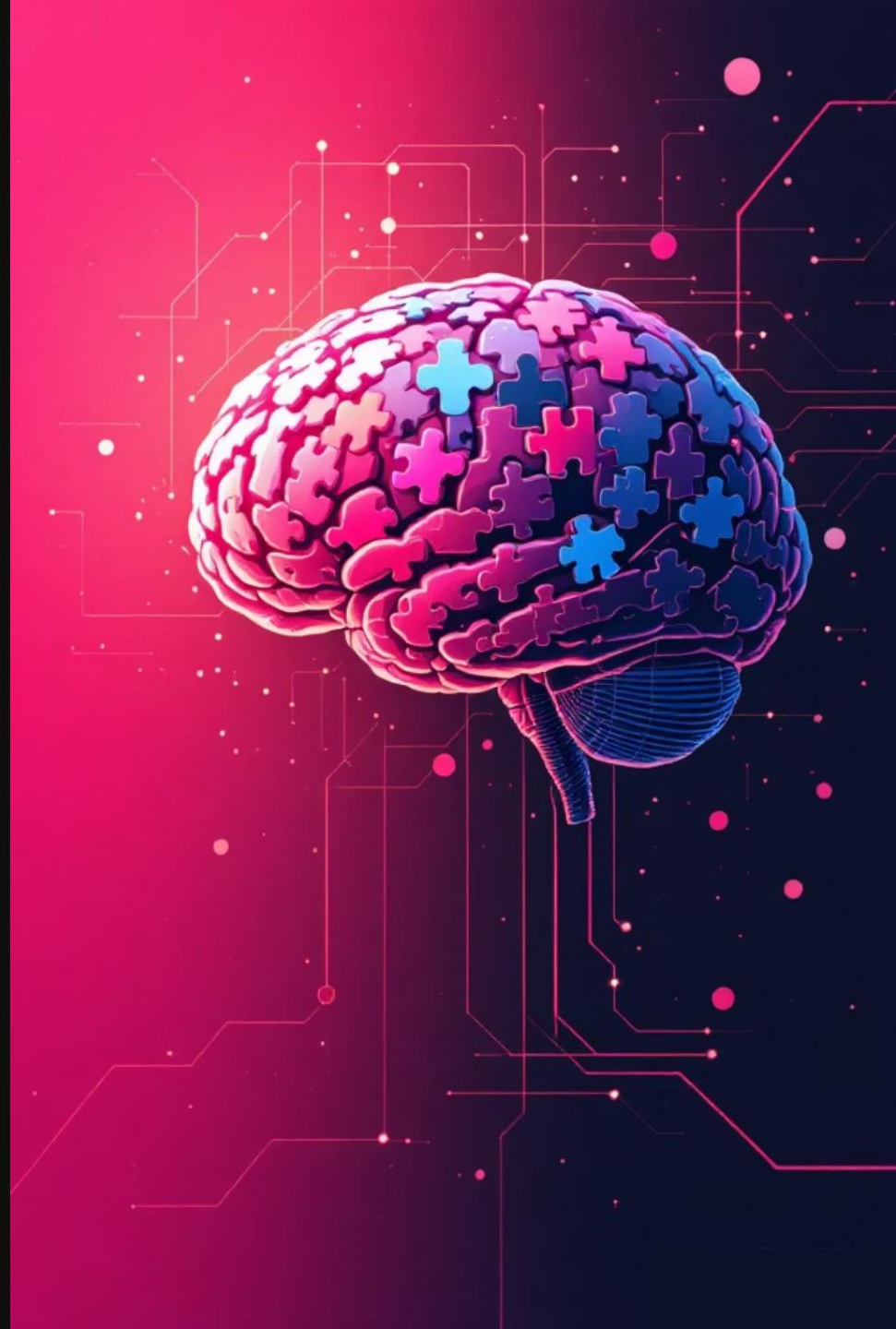
Dual-Agent Puzzle Question–Answer System

Competitive AI-Based Question Generation and Solving Framework

Presented by : Echo | AMD AI Reinforcement Learning Hackathon |

Team Members: Urvashi, Ishaanvi, Anshika, Aditi



The Challenge

Question Agent (Q-Agent)

Generates puzzle-based multiple-choice questions with logical traps, close distractors, and multi-step reasoning in strict JSON format.

Answer Agent (A-Agent)

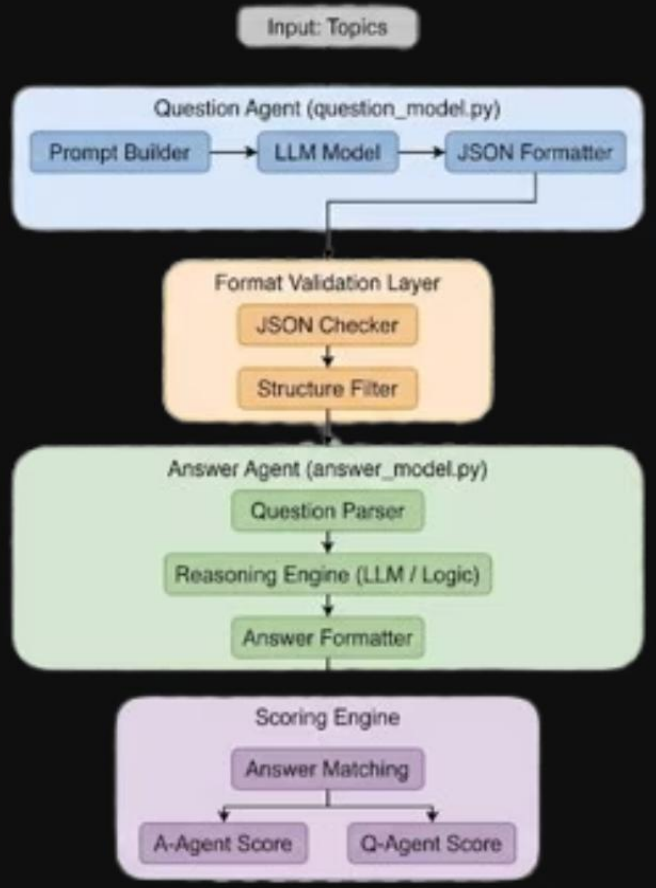
Solves generated questions through logical analysis, pattern recognition, and careful option evaluation—returning only the correct answer letter.

Competitive Scoring

Format validation filters malformed questions. Correct answers boost A-Agent score; failed questions boost Q-Agent's competitive rating.

System Architecture

From Topics to Competitive Scores



Workflow Pipeline

Our system's core architecture involves a competitive feedback loop. The **Question Agent** generates challenging questions from user input, forming a dataset. This dataset is then fed to the **Answer Agent** for evaluation, refining its solving capabilities. Simultaneously, **curated data** from these interactions is used for fine-tuning, leading to a **specialized model** that continuously improves both agents.

Question Agent

Generates questions for the system.

Curated Data

Used to fine-tune models alongside interaction outcomes.

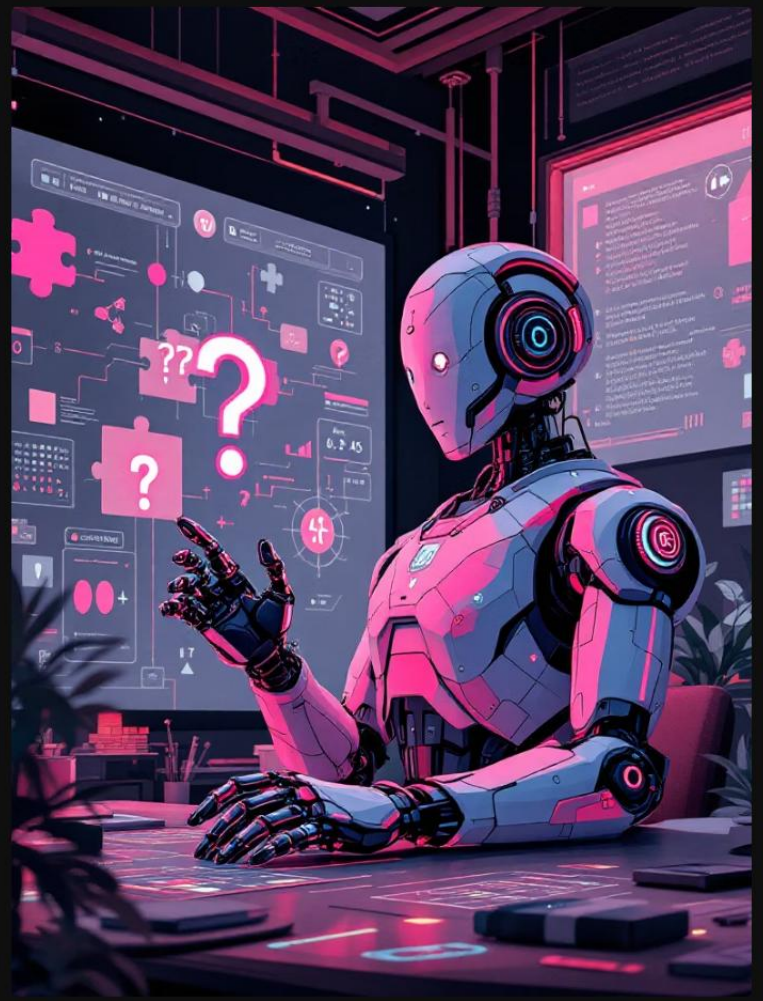
Answer Agent

Solves the generated questions.

Continuous Improvement

Ensures agents become increasingly sophisticated.

Question Agent Design



Core Capabilities

- Generates logical, puzzle-based MCQs with built-in traps
- Creates three plausible distractor options
- Maintains consistent answer mapping
- Produces valid JSON structure
- Evolves difficulty through adaptive strategy

Difficulty Strategy

- Logical inconsistencies and edge cases
- Numerical patterns requiring multi-step reasoning
- Close distractor options with subtle differences

Model Used: Qwen/Qwen3-4B

For Training purpose: Unsloth/Llama-3.1-8B-Instruct

Question JSON Format

Strict Structure Required for Validation


```
{
  "question": "I have two apples and three oranges. If I give away one apple and receive two more oranges, how many total fruits do I have?",
  "options": {
    "A": "5 fruits",
    "B": "6 fruits",
    "C": "7 fruits",
    "D": "8 fruits"
  },
  "answer": "C"
}
```


Validation Requirements

Complete question field, four distinct options labeled A-D, answer field with single uppercase letter matching one option.

Reasoning Layer

The explanation field requires clear step-by-step breakdown of calculation: initial fruits (2A+3O=5), changes (-1A+2O), final count (1A+5O=6).

Fine-Tuning Pipeline

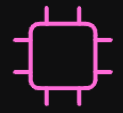
Our specialized fine-tuning pipeline optimizes model performance and resource utilization.

Unsloth Integration

Leveraged Unsloth for up to **30x faster** fine-tuning of LLMs, drastically cutting training time and computational demands.

LoRA Adapters

Employed LoRA (Low-Rank Adaptation) to efficiently fine-tune models by training only a small fraction of parameters, saving memory and compute.



ROCm bfloat16

Utilized ROCm and bfloat16 precision for superior performance and memory efficiency on AMD GPUs, maximizing hardware potential.

Gradient Checkpointing

Implemented Gradient Checkpointing to reduce memory consumption during training, enabling larger models and batch sizes.

Efficient Training

These combined techniques resulted in a streamlined, highly efficient fine-tuning pipeline, accelerating model development and iteration.

Answer Agent Design

01

Question Parsing

Extracts question text, options, and structure from JSON input

02

Logical Analysis

Identifies puzzle type, constraints, and required reasoning steps

03

Option Evaluation

Tests each option against logical constraints and calculations

04

Answer Selection

Returns only the correct option letter in minimal JSON format

Model Used: Qwen/Qwen3-4B

Scoring Mechanism

Scoring Logic in Action

After validation filters questions, both agents compete:

- A-Agent scores points for each correct answer identified
- Q-Agent scores points for questions that stump A-Agent

Formula

A-Agent Score = Correct Answers / Total Questions

Q-Agent Score = (Total Questions - Correct Answers) / Total Questions

Format-rejected questions don't count for either agent.

Format Filtering Logic

Invalid JSON rejected immediately

Missing fields (question, options, answer) rejected

Incorrect answer format (not A, B, C, or D) rejected

Disqualification if too many failures

Challenges & Innovations

Challenges Faced

- **Format Inconsistency**

  Built format filter and retry loops

- **Hallucinations**

  Added constraints and validation

- **Token Limits**

  Optimize prompting strategies

- **Ambiguous Questions**

  Reject unclear questions

Key Innovations

Self-Validation Loop

Format filter rejects malformed JSON before scoring

Adversarial Competition

Q-Agent improves from unsolved questions

Difficulty Tuning

Adjust complexity through prompt engineering

Results & Future

20	15	9	[X]
Total Questions	Format-Correct	Correct Answers	A-Agent Score
			Correct / Total Questions

[X]

Q-Agent Score

Failed Questions / Total

Future Improvements

Reinforcement Learning

Update agent prompts based on scoring feedback

Adaptive Difficulty

Scale complexity based on A-Agent success rate

Confidence Scoring

Agents report certainty; scores weighted by predicted accuracy

Multi-Agent Training

Multiple Q-Agents compete against same A-Agent

Conclusion

Built competitive dual-agent framework with format-safe generation. Demonstrated AI-to-AI evaluation through adversarial puzzle generation. System scales to arbitrary topics and difficulty levels with robust JSON handling.